

How to Build High-Quality Software

Software Engineering as Problem Solving

A guide to building quality software through values, collaboration, practices, and design.

About This Presentation

This presentation was prepared as part of **CSC 640: High Quality Software** at **Northern Kentucky University**.

Content draws from course lectures and the following primary sources:

- James Shore — *The Art of Agile Development*
- The Agile Manifesto
- Robert C. Martin — *Clean Architecture*
- Martin Fowler — *Refactoring*

Agenda

1. **Philosophy** — What software engineering is and why quality matters
2. **Collaboration** — How teams work together effectively
3. **Practices** — TDD, Pair Programming, DevOps
4. **Design** — OOP, SOLID, Patterns, Refactoring
5. **Technologies** — Frameworks, ORMs, and choosing tools
6. **Example** — A real Laravel REST API

Part 1: Philosophy

The foundation — values before methods

What is Software Engineering?

Problem solving through software

- We translate real-world problems into software solutions
- Tools change constantly — the essence doesn't
- Code is a **means**, not an end

What Makes Software Unique

- **Abstract** — You can't touch it
- **Malleable** — Easy to change (strength and danger)
- **Complex at scale** — Small programs are simple; real systems have millions of interconnected pieces
- **The solution becomes the problem** — Technical debt

Defining Success

Traditional metrics are flawed

"On time, on budget, with required features" assumes we knew what to build from the start.

Better Definition

Did we solve the problem? Did we deliver value?

- **Plans will change** — requirements shift as we learn
- **Measure outcomes**, not output
- Working software > following the original plan

The Agile Manifesto

Individuals and interactions over processes and tools

Working software over comprehensive documentation

Customer collaboration over contract negotiation

Responding to change over following a plan

The items on the right have value. The items on the left have more value.

Why Quality Matters

High-quality software:

- **Works correctly** — Does what it's supposed to do
- **Is reliable** — Keeps working under load
- **Is maintainable** — Others can understand and change it
- **Is efficient** — Uses resources reasonably

Quality enables speed

- Clean code is faster to modify
- Good tests catch bugs early
- Well-designed systems are easier to extend
- **Cutting quality feels faster short-term; slows everything long-term**

Part 2: Collaboration

People first — no amount of good code saves a dysfunctional team

What Makes Teams Effective

- **Shared understanding** — Everyone knows what they're building and why
- **Trust** — Competence trust + character trust
- **Psychological safety** — Safe to ask questions, admit mistakes, disagree
- **Clear communication** — Information flows freely

A team with great communication and average skills will outperform a team with great skills and poor communication.

Planning & Estimation

Why estimation is hard

- We're building something new
- Unknown unknowns
- Complexity and optimism bias

Iterative Planning

1. Plan a small chunk (1-2 weeks)
2. Do the work
3. Review and adjust
4. Repeat

Forecast vs. Commitment — Be honest about uncertainty

The Iron Triangle

Every project balances three things:

Constraint	Description
Scope	What you're building
Time	When it's due
Quality	How well it works

You can't fix all three.

Under pressure, scope should flex before quality. Cut features, not corners.

Managing Pressure

Sustainable Pace

- Tired people make mistakes
- Crunch borrows from the future
- Burnout loses people

Saying No (or "Yes, But")

- "Yes, but not all of it"
- "Yes, but not by then"
- "Yes, but not at this quality"

Make constraints visible so stakeholders can make informed decisions.

Culture

Psychological Safety

People feel safe to:

- Ask "dumb" questions
- Admit they don't know
- Disagree with senior people
- Point out problems

Continuous Improvement

Retrospectives: What went well? What didn't? **What will we change?**

| *Culture flows from the top. Watch what gets rewarded and punished.*

Part 3: Practices

How we work day-to-day

Test-Driven Development

Write tests first, let them guide design

Red-Green-Refactor Cycle

1. **Think** — Identify smallest increment of behavior
2. **Red** — Write a failing test
3. **Green** — Write just enough code to pass
4. **Refactor** — Improve the code
5. **Repeat** — In under 5 minutes

TDD works like double-entry bookkeeping: express intent twice, in different ways.

Why TDD Works

- **Instant feedback** — Mistakes found in seconds, not hours
- **Better design** — Forced to think about usage before implementation
- **Confidence to refactor** — Tests catch errors
- **Near-zero debugging** — When something breaks, there's almost nothing to check

Types of Tests

Type	Speed	Quantity
Unit tests	Hundreds/second	Many
Integration tests	Handful/second	Few
End-to-end tests	Slow, brittle	Minimal

Pair Programming

Two minds, one keyboard, better code

How It Works

- **Driver** — Types, focuses on tactics
- **Navigator** — Thinks ahead, focuses on strategy
- Switch roles frequently (every 30 min)

Benefits

- 15% more effort → 15% fewer defects
- Knowledge spreads naturally
- More time in flow
- Positive peer pressure for good habits

Pair Programming Tips

- Sit side by side, monitor visible to both
- Think out loud, discuss assumptions
- Switch partners several times per day
- Never assign partners — pairs form naturally

Common Questions

"Isn't it wasteful?" — Only one keyboard, but there's more to programming than typing.

"What about introverts?" — It feels uncomfortable at first. Try it for a month before judging.

DevOps

Breaking down walls between Dev, Ops, and Security

The Problem

- Developers build, then hand off
- Operations is distant
- Security reviews happen at the end
- **Walls create problems**

The Solution

Cross-functional teams that include (or collaborate with) ops and security from the beginning.

Building for Operation

Shift Left

Think about security and operations from the start, not the end.

- **Threat modeling** — Diagram architecture, brainstorm what can go wrong
- **Configuration** — Separate from code; deploy same code everywhere
- **Secrets** — Secure procedures for passwords and API keys
- **Logging & Monitoring** — What will someone need at 3am?

CI/CD

- **Continuous Integration** — Integrate frequently, automated build & test
- **Continuous Deployment** — Every passing change deploys automatically

Part 4: Design

How we think about code

OOP Principles (APIEC)

Principle	Description
Abstraction	Expose only what matters, hide the rest
Polymorphism	Different objects respond to same message differently
Inheritance	Derive from another class ("is-a" relationship)
Encapsulation	Bundle data with operations, control access
Composition	Build complex objects from simpler ones ("has-a")

"Favor composition over inheritance"

SOLID Principles

Letter	Principle	Summary
S	Single Responsibility	One reason to change
O	Open/Closed	Open for extension, closed for modification
L	Liskov Substitution	Subtypes must be substitutable
I	Interface Segregation	Don't force unused dependencies
D	Dependency Inversion	Depend on abstractions, not concretions

Guidelines, not laws. Following them blindly can create unnecessary complexity.

Design Patterns

Proven solutions to recurring problems

Categories

- **Creational** — Factory, Builder, Singleton
- **Structural** — Adapter, Facade, Decorator, Proxy
- **Behavioral** — Observer, Strategy, Command, Iterator

A Word of Caution

| *Patterns are tools, not goals. Don't force a pattern where it doesn't fit.*

Learn patterns to recognize when they help — not to show off.

Refactoring

Improving structure without changing behavior

How to Refactor

- Small steps (seconds to minutes)
- Tests pass after each step
- String many small refactorings together

Code Smells

- **Divergent Change** — One class changes for multiple reasons
- **Shotgun Surgery** — One change touches multiple classes
- **Primitive Obsession** — High-level concepts as primitives
- **Data Clumps** — Variables that travel together

Without refactoring, technical debt eventually overwhelms you.

Part 5: Technologies

What we build with

High-Level Frameworks

Abstraction that enables productivity

What Frameworks Provide

- **Structure** — Conventions, less decision fatigue
- **Solved Problems** — Security, auth, validation
- **Ecosystem** — Libraries, plugins, community

The Trade-Off

Frameworks impose constraints. Choose frameworks whose opinions match your problem.

Why Frameworks Exist

Every framework started as someone's frustration:

- Rails — tired of boilerplate
- React — complex UI state
- Laravel — PHP without structure

Framework vs. Library

Library	Framework
You call it	It calls you
You're in control	It's in control
Pick what you need	Fill in the blanks

Before adopting a tool, ask: What problem does this solve? Do I have that problem?

ORMs & Abstraction Layers

Simplifying data access

```
// Without ORM
$result = $db->query("SELECT * FROM users WHERE id = 1");
$user = new User($result['id'], $result['name']);

// With ORM
$user = User::find(1);
```

Benefits

- Query generation
- Result mapping
- Relationship traversal
- Automatic SQL injection prevention

Danger: Learn to read the SQL your ORM generates.

Choosing the Right Tool

Questions to Ask

1. What problem does it solve?
2. Do I have that problem?
3. What's the learning curve?
4. What's the community like?
5. What are the escape hatches?

The Boring Technology Rule

Every team has a limited budget for complexity. Spend it on your business problems, not exciting new databases.

PostgreSQL, Linux, Python aren't exciting. They're reliable.

Part 6: Example

The GiftList API — See it all in action

Why Laravel for GiftList?

Without a framework:

- Write authentication from scratch
- Manually handle database connections
- Build routing system
- Sanitize inputs manually

With Laravel:

- `Route::middleware('auth:sanctum')` — one line for auth
- `User::find($id)` — one line for queries
- `Route::apiResource('gifts', GiftController::class)` — auto-generates REST routes

The framework handles plumbing so you focus on business logic.

Eloquent ORM in Action

Plain PHP

```
$stmt = $pdo->prepare("SELECT * FROM gifts WHERE id = ?");  
$stmt->execute([$id]);  
$row = $stmt->fetch();  
// ... manual mapping
```

Eloquent

```
$gift = Gift::find($id);  
$recipient->gifts; // Relationship traversal
```

90% reduction in database code

API Architecture

MVC Pattern

- **Models** — Data structure and relationships
- **Controllers** — Handle requests, return responses
- **Routes** — Map URLs to controller methods

RESTful Endpoints

Method	Endpoint	Action
GET	/api/gifts	List all
POST	/api/gifts	Create
GET	/api/gifts/{id}	Get one
PUT	/api/gifts/{id}	Update
DELETE	/api/gifts/{id}	Delete

Why Conventional Architecture Wins

The GiftList API isn't impressive because it's complex.

It's useful because it's **simple and follows established patterns**:

- New developers understand it immediately
- Standard tools work out of the box
- Changes are isolated to appropriate layers
- Testing is straightforward

Choose frameworks whose opinions match your problem. Let conventions accelerate development.

Summary

Key Takeaways

Philosophy

- Software engineering is problem solving
- Quality enables sustainable speed

Collaboration

- People > processes
- Make constraints visible

Practices

- TDD: Red → Green → Refactor
- Pair programming shares knowledge
- DevOps breaks down walls

Key Takeaways (continued)

Design

- APIEC: Abstraction, Polymorphism, Inheritance, Encapsulation, Composition
- SOLID: Guidelines for maintainable code
- Refactor constantly in small steps

Technologies

- Frameworks solve developer problems
- Choose boring technology
- Understand what your tools do underneath

Example

- Conventions accelerate development
- Simple > clever

Thank You

How to Build High-Quality Software

Questions?

References

- James Shore — *The Art of Agile Development*
- The Agile Manifesto — agilemanifesto.org
- Martin Fowler — *Refactoring*
- Gang of Four — *Design Patterns*
- Robert C. Martin — *Clean Architecture*
- Dan McKinley — "Choose Boring Technology"

Full documentation: enesabd.github.io/HQS-SWE-Documentation